INHA UNIVERSITY TASHKENT
DEPARTMENT OF CSE & ICE
SPRING SEMESTER 2024

SOC 2040 - SYSTEMS PROGRAMMING

LAB ASSIGNMENT 5

INSTRUCTIONS :

- **ALL LAB ASSIGNMENTS ARE TO BE COMPLETED IN GROUPS OF 4 TO 6 STUDENTS.**

- **LAB ASSIGNMENT REPORT SHOULD BE PREPARED USING THE LAB ASSIGNMENT 5 REPORT TEMPLATE PROVIDED AT THE ECLASS PORTAL. NO OTHER REPORT FORMAT WILL BE ACCEPTED AND WILL GET A GRADE 0**

- **HARDCOPY OF THE LAB ASSIGNMENT5 REPORT MUST BE SPIRALLY BOUNDED. ONLY SPIRALLY BOUNDED HARD COPY OF THE REPORT IS ACCEPTED AND ANY OTHER FORM IS REJECTED OUTRIGHTLY.**

- **LAST DATE FOR UPLOADING THE LAB ASSIGNMENT5 REPORT AT THE ECLASS PORTAL IS WEDNESDAY, 15TH MAY 2024.**

- **EVERY MEMBER OF THE TEAM MUST UPLOAD THE SOFTCOPY OF THE REPORT AT THE ECLASS PORTAL.**
- **NO GRADES WILL BE GIVEN IF SOFTCOPY OF THE REPORT IS NOT UPLOADED AT THE ECLASS.**

- **NO EMAIL SUBMISSION IS ACCEPTED – IT WILL BE OUT-RIGHTLY REJECTED AND GIVEN A GRADE 0.**

- **ONE HARD COPY OF THE LAB ASSIGNMENT5 REPORT OF EACH GROUP LEADER SHOULD BE HANDED IN AT THE OFFICE BY THE GROUP LEADER ON THURSDAY, 16TH MAY 2024 BETWEEN 1.30PM & 4.30PM.**

- **HARDCOPY REPORT OF ONLY THE LEADER OF THE APPROVED TEAM GIVEN AT THE ECLASS PORTAL WILL BE ACCEPTED AT MY OFFICE ON THURSDAY, 16TH MAY 2024 BETWEEN 1.30PM & 4.30PM.**

- **PLEASE NOTE THAT SOFTCOPY OF ALL MEMBERS INCLUDING THE TEAM LEADER OF ONLY THE APPROVED TEAM UPLOADED AT THE ECLASS WILL BE CONSIDERED FOR GRADING.**
- **STUDENTS WHOSE NAMES NOT IN THE APPROVED LIST GIVEN AT THE ECLASS PORTAL WILL BE GIVEN GRADE 0.**

- **LATE SUBMISSION WILL NOT BE ACCEPTED AND GIVEN A GRADE 0.**

- **PRINTOUTS CAN BE TAKEN ON BOTH SIDES OF THE PAPER.**
- **PRINTOUTS TAKEN MUST BE LEGIBLE AND PERFECTLY READABLE – OTHERWISE IT WILL NOT BE GRADED AND GIVEN A GRADE 0 OUTRIGHTLY.**

- **IN THE REPORT, FOR EACH C PROGRAMMING QUESTION YOU NEED TO PROVIDE THE ALGORITHM, PROGRAM AND SCREENSHOTS OF THE PROGRAM ENTERED, INPUTS GIVEN & RESULTS OBTAINED**

- **EVERY MEMBER MUST EXECUTE ALL PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, INPUTS GIVEN AND RESULTS**

- **ALL FILES CONTAINING 'C' PROGRAMS WRITTEN AND EXECUTED FOR ALL QUESTIONS SHOULD BE UPLOADED AT THE E-CLASS PORTAL ALONG WITH THE LAB ASSIGNMENT 5 REPORT.**
- **YOU ARE REQUIRED TO PUT ALL THE FILES IN A FOLDER AND GIVE THE FOLDER NAME - YOUR ID FOLLOWED BY SPLABASSIGNMENT5(EXAMPLE:U2210100_SPLABASSIGNMENT5).**

- **LATE SUBMISSIONS ARE NOT ENTERTAINED, ADHERE TO THE DEADLINE STRICTLY.**
- **READ THE QUESTIONS CORRECTLY & CAREFULLY**

SPECIFIC INSTRUCTIONS REGARDING SP LAB ASSIGNMENT5:

- **EVERY MEMBER MUST EXECUTE ALL PROGRAMS GIVEN & DEVELOPED ON HIS/HER COMPUTER SYSTEM FOR ALL QUESTIONS 1 TO 8 AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, COMMANDS GIVEN, INPUTS GIVEN AND RESULTS.**

- **FOR QUESTIONS 7 AND 8, THE PROGRAM WRITING CAN BE SHARED AMONG THE MEMBERS, BUT EVERY MEMBER MUST EXECUTE THE PROGRAMS DEVELOPED ON HIS/HER COMPUTER SYSTEM**

- **READ THE QUESTIONS CAREFULLY AND ALSO UNDERSTAND THE PROGRAMS GIVEN CORRECTLY SO THAT YOU CAN CLEARLY DESCRIBE IN WORDS THE SEQUENCE OF OPERATIONS CARRIED OUT BY EACH OF THE PARENT AND CHILD PROCESSES CREATED AND SEQUENCE OF THEIR TERMINATIONS.**

- **EVERY MEMBER MUST DRAW THE PROCESS GRAPHS BY HAND FOR ALL THE PROGRAMS GIVEN IN QUESTION 1 (fork1.c to fork12.c) ON A4 SIZE WHITE PAPER, THAT IS, LAB REPORT PAPER.**

- **ALL TEAM MEMBERS MUST SUBMIT ORIGINAL OF THE HAND DRAWN PROCESS GRAPH SHEETS TO THE TEAM LEADER TO BE INCLUDED IN THE HARD COPY OF THE LAB ASSIGNMENT5 REPORT WITH THEIR NAMES, STUDENT IDs &**

SIGNATURES ENTERED ON THESE SHEETS. SCANNED COPY OF THESE PROCESS GRAPH SHEETS SHOULD BE INCLUDED IN THEIR RESPECTIVE REPORTS TO BE UPLOADED AT THE ECLASS PORTAL.

- **EVERY MEMBER MUST ALSO SUBMIT A COPY OF THE SCREEN SHOTS OF THE RESULTS OF ALL THE PROGRAMS (FOR QUESTIONS 1 TO 8) EXECUTED ON HIS/HER COMPUTER TO THE TEAM LEADER TO BE INCLUDED IN THE HARDCOPY OF THE LAB ASSIGNMENT5 REPORT OF THE TEAM LEADER.**
- **LAST DATE FOR UPLOADING THE LAB ASSIGNMENT5 REPORT AT THE ECLASS PORTAL IS WEDNESDAY, 15TH MAY 2023.**
- **ONE HARD COPY OF THE LAB ASSIGNMENT REPORT OF EACH GROUP LEADER SHOULD BE HANDED IN AT THE OFFICE BY THE GROUP LEADER ON THURSDAY, 16TH MAY 2023 BETWEEN 1.30PM & 4.30PM.**

- **ALL FILES CONTAINING 'C' PROGRAMS WRITTEN AND EXECUTED FOR ALL QUESTIONS SHOULD BE UPLOADED AT THE E-CLASS PORTAL ALONG WITH THE LAB ASSIGNMENT 5 REPORT.**
- **- YOU ARE REQUIRED TO PUT ALL THE FILES IN A FOLDER AND GIVE THE FOLDER NAME - YOUR ID FOLLOWED BY SPLABASSIGNMENT5(EXAMPLE:U2210100_SPLAB ASSIGNMENT5).**
- **LATE SUBMISSIONS ARE NOT ENTERTAINED, ADHERE TO THE DEADLINE STRICTLY.**
- **READ THE QUESTIONS CORRECTLY & CAREFULLY**

QUESTIONS

ALL REQUIRED PROGRAMS ARE AVAILABLE IN THE FOLDER sp5programs GIVEN AT THE ECLASS PORTAL

EVERY MEMBER MUST EXECUTE ALL PROGRAMS Q1) TO Q8) DEVELOPED ON HIS/HER COMPUTER SYSTEM AND REQUIRED TO PROVIDE SCREENSHOTS OF PROGRAM ENTERED, COMMANDS GIVEN, INPUTS GIVEN AND RESULTS

1. You are given a set of programs with file names **fork1.c to fork12.c** (**12 programs**) to study the process creation and termination. You are required to go through each of these programs and do the following:

   a) Describe in detail the sequence of operations carried out by each of the parent and child processes created and sequence of their terminations.

   b) Draw the corresponding **Process Graph** for each program clearly showing the parent and child processes execution from creation to termination.

   c) Run each program separately and provide all the outputs of the parent and child processes and provide your analysis on these outputs.

FOR ANSWERS, INDICATE CLEARLY QUESTION NUMBER 1 & SUBSECTION NUMBER a), b), c) IN YOUR REPORT

PROVIDE SCREENSHOTS OF ALL THE PROGRAMS ENTERED, COMMANDS GIVEN AND THE OUTPUTS OBTAINED

2. You are given the following **five programs** to show the creation of Zombie processes :
zombie_child1.c, zombie_childn.c, zombie_childs.c, zombie_childsleep.c, zombie_childpause.c (5 Programs)

   a) Run each program separately and provide all the outputs of the parent and child processes and provide your analysis on these outputs.

   For programs **zombie_childn.c, zombie_childs.c, zombie_childsleep.c, zombie_childpause.c , every member must take 6 to 9 different values for n and execute the programs.**

b) Identify the child processes in the Zombie states by running the ps -la command and provide the screenshots with proper explanation

c) Also use appropriate commands to kill all the Zombie processes and provide necessary screen shots to show that these processes no longer exist.

FOR ANSWERS, INDICATE CLEARLY QUESTION NUMBER 2 & SUBSECTION NUMBER a), b), c) IN YOUR REPORT

PROVIDE SCREENSHOTS OF ALL THE PROGRAMS ENTERED, INPUTS GIVEN AND THE OUTPUTS OBTAINED

3. You are given the following **six programs** to show how parent process reaps child processes thereby avoiding creation of zombie processes :

waitonchild1.c, simplewait_childao.c, waitpid_childao.c, waitpid_childco.c, waitpid_childsleepco.c, waitclktchild.c (6 Programs)

a) Run each program separately and provide all the outputs of the parent and child processes and provide your analysis on these outputs.

b) Also comment on the order in which child processes are reaped by the parent process.

FOR ANSWERS, INDICATE CLEARLY QUESTION NUMBER 3 & SUBSECTION NUMBER a), b) IN YOUR REPORT

PROVIDE SCREENSHOTS OF ALL THE PROGRAMS ENTERED AND THE OUTPUTS OBTAINED

4. You are given the following **eleven programs** to show the importance of signals and how signals are handled in a process :

signalhandler1.c, signalhandler2.c, signalhandler3.c, signalhandler4.c, onesighandler.c, multsighandler.c, signalbeep.c, signalint.c, signalquit.c signalkill.c, signalkillchildn.c

a) Run each program separately and provide the outputs

b) Provide your analysis on these outputs.

FOR ANSWERS, INDICATE CLEARLY QUESTION NUMBER 4 & SUBSECTION NUMBER a), b) IN YOUR REPORT

PROVIDE SCREENSHOTS OF ALL THE PROGRAMS ENTERED AND THE OUTPUTS OBTAINED

5. You are given the following **two programs** to show the importance of Nonlocal jumps and how setjmp and longjmp are handled in a process :

nonlocaljmp1.c, nonlocaljmp2.c

a) Run each program separately and provide the outputs.

b) Provide your analysis on these outputs.

FOR ANSWERS, INDICATE CLEARLY QUESTION NUMBER 5 & SUBSECTION NUMBER a), b) IN YOUR REPORT

PROVIDE SCREENSHOTS OF ALL THE PROGRAMS ENTERED AND THE OUTPUTS OBTAINED

6. You are given the following **program threadvshare.c** showing how variables are shared in threads.

a) Run this program and provide the outputs produced by each thread.

b) Also identify clearly the variables shared and not shared between the threads.

FOR ANSWERS, INDICATE CLEARLY QUESTION NUMBER 6 & SUBSECTION NUMBER a), b) IN YOUR REPORT

PROVIDE SCREENSHOTS OF ALL THE PROGRAMS ENTERED AND THE OUTPUTS OBTAINED

7. You are given the following **program badcntn.c** to illustrate the effect of improper synchronization in threads**.**

   a) Run this program several times with the same inputs (number of iterations and number of threads specified as command line arguments For example - **$badcntn 20000 2000**) and identify the thread synchronization problem (shared variable involved) for getting different answers at different runs.

   b) Draw a Process Graph showing the concurrent execution of the above instructions in the assembly code executed concurrently by the threads thread1 and thread2.

   c) Identify the critical region and the instructions in that region.

   d) Draw the unsafe region in the process graph.

   e) Show atleast EIGHT Safe Trajectories in the progress graph which lead to proper thread synchronization.

   f) Show atleast EIGHT Unsafe Trajectories in the progress graph which lead to improper thread synchronization.

   g) Modify the above program (call it **goodcntn.c**) using semaphores to provide proper synchronization (use semaphores **P(&mutex) and V(&mutex)** to control access to **cnt** shared variable) so that same results will be obtained for different runs of the program with the same input.

FOR ANSWERS, INDICATE CLEARLY QUESTION NUMBER 7 & SUBSECTION NUMBER a), b), c), d), e), f), g) IN YOUR REPORT

PROVIDE SCREENSHOTS OF ALL THE PROGRAMS ENTERED AND THE OUTPUTS OBTAINED

8. Using the given shell example program **shellcmd.c** as a starting point, write a shell program that supports job control.

   Your shell should have the following features:

   - Shell prompt is <> (less than symbol followed by greater than symbol)
   - The command line typed by the user consists of a name and zero or more arguments, all separated by one or more spaces.
   - If name is a built-in-linux-command, the shell handles it immediately and waits for the next command line.
   - Otherwise, the shell assumes that name is an executable file, which it loads and runs in the context of an initial child process (job).
   - The process group ID for the job is identical to the PID of the child.
   - Each job is identified by either a process ID (PID) or a job ID (JID), which is a small arbitrary positive integer assigned by the shell.
   - JIDs are denoted on the command line by the prefix '%'.
   - For example,"%5" denotes JID 5, and "5" denotes PID 5.
   - If the command line ends with an ampersand, then the shell runs the job in the background.
   - Otherwise, the shell runs the job in the foreground.
   - Typing ctrl-c (ctrl-z) causes the shell to send a SIGINT(SIGTSTP) signal to every process in the foreground process group.
   - The jobs built-in command lists all background jobs.
   - The bg <job> built-in command restarts <job> by sending it a SIGCONT signal, and then runs it in the background.
   - The <job> argument can be either a PID or a JID.
   - The fg <job> built-in command restarts <job> by sending it a SIGCONT signal, and then runs it in the foreground.
   - The shell reaps all of its zombie children.
   - If any job terminates because it receives a signal that was not caught, then the shell prints a message to the terminal with the job's PID and a description of the offending signal.

   A Sample Shell Session for this Question given in below (SEE NEXT PAGE)

Sample Shell Session for the Question

$ ./shellcmd	Run your shell program
<> bogus	
bogus: Command not found	Execve can't find executable
<> foo 10	
Job 5035 terminated by signal: Interrupt	User types ctrl-c
<> foo 100 &	
[1] 5036 foo 100 &	
<> foo 200 &	
[2] 5037 foo 200 &	
<> jobs	
[1] 5036 Running foo 100 &	
[2] 5037 Running foo 200 &	
<> fg %1	
Job [1] 5036 stopped by signal: Stopped	User types ctrl-z
<> jobs	
[1] 5036 Stopped foo 100 &	
[2] 5037 Running foo 200 &	
<> bg 5035	
5035: No such process	
<> bg 5036	
[1] 5036 foo 100 &	
<> /bin/kill 5036	
Job 5036 terminated by signal: Terminated	
<> fg %2	Wait for fg job to finish.
<> quit	
$	Back to the Unix shell

PROVIDE SCREENSHOTS OF ALL THE PROGRAMS ENTERED AND THE OUTPUTS OBTAINED

****